

# PNP Formal Reconstruction Report

Compiled Lean theorem inventory and current claim boundary

Coordinate PNP-FORMAL-RECONSTRUCTION-STATUS-2026-07-14-37

**The repository does not currently establish**  
 **$P = NP$ .**

Public theorem emission is disabled. A direct finite raw-machine verifier now proves concrete CNF-SAT membership in NP with an explicit polynomial bound. It does not prove CNF-SAT in P, NP-hardness, NP-completeness, or  $P = NP$ . The general charged pipeline has not been linked to the raw machine kernel, its reviewed activation fingerprints are unset, and no root theorem exists. The abstract string-handle bridge is never an activation source.

|                                           |              |
|-------------------------------------------|--------------|
| Compiled declarations                     | 6007         |
| Theorem-kind declarations                 | 2514         |
| Assumption-free theorem-kind declarations | 2401         |
| Project-specific axioms                   | 4            |
| Concrete publication gate passed          | <b>false</b> |

Inventory SHA-256: 7e6aac322da8d6bb8db813217daf6411fae937cd101f3b6e960cefeb56330590

Generated from the pinned compiled Lean environment, not from source-text parsing.

Contents

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Current authority and claim boundary</b>     | <b>2</b>  |
| <b>2</b> | <b>Concrete publication gate</b>                | <b>2</b>  |
| 2.1      | Why the abstract bridge is ineligible . . . . . | 3         |
| <b>3</b> | <b>Formal milestone ledger</b>                  | <b>3</b>  |
| <b>4</b> | <b>Compiled inventory summary</b>               | <b>8</b>  |
| <b>5</b> | <b>Remaining formal blockers</b>                | <b>9</b>  |
| <b>6</b> | <b>Historical report provenance</b>             | <b>9</b>  |
| <b>7</b> | <b>Verification</b>                             | <b>10</b> |

## 1 Current authority and claim boundary

This report is the current repository report surface. Its factual theorem inventory comes from the compiled PNP environment under `leanprover/lean4:v4.31.0`. The exporter enumerates `Environment.constants`, resolves each originating module, and calls `Lean.collectAxioms` for every exported project declaration. The committed inventory is `status/LEAN_THEOREM_INVENTORY.json`; the public mirror is byte-identical.

The target remains  $P = NP$ , but target wording—including the new inactive definition `PNP.Main.ConcretePEqualsNP`—is not theorem evidence. JavaScript acceptance, Boolean fields, JSON strings, historical release records, report prose, and external review cannot activate a theorem. Only the separately derived concrete publication gate may control theorem emission, and that gate is currently false.

The completed direct CNF verifier consumes the literal canonical pair of an encoded formula and assignment certificate. Lean proves universal acceptance equivalence, rejection equivalence, and absence of timeout at its explicit polynomial fuel bound, and constructs a proof-bearing `PolynomialTimeVerifier CNFSAT`. Thus `PNP.Concrete.CNFSAT` is in the concrete NP class. This is a verifier theorem, not a deterministic polynomial-time SAT algorithm.

| Machine field                               | Current value                 |
|---------------------------------------------|-------------------------------|
| <code>mathematicalTheoremEstablished</code> | <b>false</b>                  |
| <code>publicTheoremEmissionAllowed</code>   | <b>false</b>                  |
| <code>finalTheoremReady</code>              | <b>false</b>                  |
| <code>publicTheoremStatement</code>         | null                          |
| <code>rootLeanTheorem</code>                | <code>PNP.Main.p_eq_np</code> |
| <code>rootLeanTheoremPresent</code>         | <b>false</b>                  |

## 2 Concrete publication gate

The compatibility entry is `PNP.Main.p_eq_np`; a matching name alone is insufficient. The separate publication target is `PNP.Main.ConcretePEqualsNP`. Eligibility also requires the exact theorem type, reviewed kernel fingerprints, and a tightly fixed Lean-standard axiom closure. The target is present as an inactive definition backed by finite charged-pipeline semantics; its presence is not a proof, and the compiler/refinement link to the raw machine kernel remains an explicit blocker for the general charged-pipeline target. The direct CNF verifier is one closed raw-machine instance; it does not supply that general link.

The expected fingerprints are intentionally unset. This is a fail-closed migration gate, not a missing runtime input. A verifier must reject any attempt to set `passed=true` while a fingerprint is null, while the concrete model is ineligible, or while any other subcheck is false.

| Gate subcheck                                               | Result       |
|-------------------------------------------------------------|--------------|
| <code>standardComplexityModelEligible</code>                | <b>true</b>  |
| <code>concreteTargetPresent</code>                          | <b>true</b>  |
| <code>concreteTargetIsDefinition</code>                     | <b>true</b>  |
| <code>concreteTargetKernelTypeFingerprintConfigured</code>  | <b>false</b> |
| <code>concreteTargetKernelTypeFingerprintMatches</code>     | <b>false</b> |
| <code>concreteTargetKernelValueFingerprintConfigured</code> | <b>false</b> |
| <code>concreteTargetKernelValueFingerprintMatches</code>    | <b>false</b> |
| <code>compatibilityRootPresent</code>                       | <b>false</b> |
| <code>compatibilityRootIsTheorem</code>                     | <b>false</b> |

| Gate subcheck                                    | Result       |
|--------------------------------------------------|--------------|
| compatibilityRootHasExactConcreteType            | <b>false</b> |
| compatibilityRootKernelTypeFingerprintConfigured | <b>false</b> |
| compatibilityRootKernelTypeFingerprintMatches    | <b>false</b> |
| axiomClosureFingerprintConfigured                | <b>false</b> |
| axiomClosureFingerprintMatches                   | <b>false</b> |
| sourceClosureFingerprintConfigured               | <b>false</b> |
| sourceClosureFingerprintMatches                  | <b>false</b> |
| axiomClosureUsesOnlyLeanStandardAllowlist        | <b>false</b> |

Allowed foundation axioms are fixed to `Classical.choice`, `Quot.sound`, and `propext`. Project axioms, `sorryAx`, or any unknown axiom make the gate fail. The four currently disclosed project axioms are:

- `PNP.CheckPCCPackexp`
- `PNP.GeneratePCCPack`
- `PNP.LockedNANDThreshold`
- `PNP.ResidualBandExactMinimization`

## 2.1 Why the abstract bridge is ineligible

The current `PNP.PEqualsNP` abbreviation compares witness classes whose language and algorithm structures carry string names or code handles rather than concrete execution semantics and proved polynomial bounds. Consequently, even a kernel-clean theorem of that abstract type would not establish the standard P-versus-NP statement. It is compatibility information only.

The separate `PNP.Concrete.PEqualsNP` target uses proof-bearing P and NP witnesses, canonical input/certificate pairing, finite program syntax, and charged polynomial runtime and output bounds. This is a substantive formalization milestone, but it is not yet proved equivalent to execution in the raw machine kernel. SAT completeness, SAT in P, and the root theorem also remain absent.

## 3 Formal milestone ledger

Each earned row requires exact compiled names and theorem kinds, empty axiom closures, per-name domain-separated kernel-type SHA-256 matches for 254 detailed candidates, and the pinned whole-Lean source closure. Human-readable scope remains conservative: type or source drift revokes credit.

| Milestone                                 | Status                     | Exact scope                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Boundary                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Concrete machine and cost kernel          | formalized-foundation-only | <p>One literal finite four-stage pipeline handles every raw bitstring. The total framer uses exactly 4 work steps on empty input, <math>4 * k * k + 9 * k + 7</math> for complete two-bit cells, and <math>4 * k * k + 9 * k + 5</math> when the final cell is partial; its compiled raw cost is bounded by <math>6 * m * m + 39 * m + 75</math>. Every supplied exact n-step target run lifts to exactly <math>3 * n</math> work steps. PipelineCompiler preserves one raw target's verdict and ordinary output at an explicit external polynomial. PipelineSequentialCompiler composes two raw targets in one literal finite table, passes either first verdict onward, preserves the second verdict and output, retains stuck-first timeout, and proves <math>R(m) = \text{PipelineRaw}(p)(m) + 6 + \text{PipelineRaw}(q)(m + p(m) + 1)</math>. PipelineRefinement recursively applies that compiler to every FunctionProgram composition and DecisionProgram precomposition node. Its 16 audited declarations have empty axiom closure, and PolynomialTimeDecider.compileToMachine exposes the complete tree as one raw polynomial-time machine.</p> | <p>This closes the concrete complexity machine-link blocker only. It does not construct a deterministic polynomial-time CNF-SAT decider or an NP-completeness reduction. CNF-SAT in P, NP-completeness, and <math>P = NP</math> are not established; PNP.Main.p_eq_np remains absent and the publication gate remains false.</p> |
| Concrete P, NP, and polynomial reductions | formalized                 | <p>Bitstring languages, finite charged decision/verifier/function pipelines grounded in concrete machines, polynomial certificate/runtime/output bounds, recursively compiled exact raw-machine refinements, many-one reductions, and the NP-complete-in-P implication.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | <p>The recursive refinement compiler closes the charged-to-raw machine link, but it does not construct raw paired verifiers beyond the existing CNF-SAT membership witness, prove CNF-SAT is in P or NP-complete, or establish the publication root theorem.</p>                                                                 |

| Milestone                                          | Status                        | Exact scope                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Boundary                                                                                                                                                                                                                                                                              |
|----------------------------------------------------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Concrete universal CNF-SAT verifier                | formalized-np-membership-only | An unconditional formula-grammar outcome, universal bounded work outcome, exact accept/reject semantics, no-timeout theorem, literal finite-machine paired verifier, and CNF-SAT membership in NP.                                                                                                                                                                                                                                                                                | This proves CNF-SAT membership in NP only; it does not prove CNF-SAT is in P, NP-completeness, or $P = NP$ .                                                                                                                                                                          |
| Concrete Cook-Levin dimensions and variable layout | formalized-foundation-only    | Fuel padding preserves designated halts and stuck timeouts; every literal machine state is below an explicit finite ceiling; the input, time, and tape dimensions are executable NatPolynomial expressions; and tape-symbol, head, state, certificate-bit, and certificate-length variables occupy proved disjoint in-range blocks. Elementary at-least-one and pair-exclusion CNF clauses have exact propositional semantics. All 20 reviewed theorems have empty axiom closure. | This is the finite layout substrate only. It does not yet construct a transition tableau, prove tableau soundness or completeness, compile a formula emitter, define a polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or $P = NP$ .                |
| Concrete fixed-certificate execution tableau       | formalized-foundation-only    | For one fixed source input, certificate, finite-rule machine, and fuel budget, the canonical raw execution trace has exactly $\text{fuel} + 1$ configurations. Intrinsic first-match transition validity is equivalent to equality with that trace, every valid endpoint equals the ordinary run endpoint, and accept/reject/timeout verdicts agree exactly with boundedDecide. All 30 declarations and eight reviewed theorem types have empty axiom closure.                    | This is the semantic fixed-certificate tableau only. It does not yet quantify a variable certificate, encode tape windows or tableau rows as CNF, compile a formula emitter, define a polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or $P = NP$ . |

| Milestone                            | Status                     | Exact scope                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Boundary                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Uniform bounded verifier tableaux    | formalized-foundation-only | For an arbitrary proof-bearing verifier in the concrete NP model and one source input, the complete finite decision pipeline is compiled to one raw machine. Every certificate inside the explicit certificate polynomial shares one answer-independent raw fuel obtained by evaluating the compiled raw-time polynomial at the maximum encoded input size. Per-certificate fuel is below that bound, padding follows a proved halt, exact verdicts are preserved, and language membership is equivalent to an existential bounded accepting semantic tableau. All 41 declarations and nine reviewed theorem types have empty axiom closure. | This is a uniform semantic verifier-tableau equivalence only. It does not encode configurations, transition windows, or variable-length certificates as Boolean clauses; emit CNF; prove emitter size or runtime polynomials; define a polynomial reduction to CNFSAT; or establish CNFSAT NP-completeness, CNFSAT in P, or $P = NP$ .  |
| Finite local Cook-Levin CNF compiler | formalized-foundation-only | Width-indexed literals materialize exact Boolean assignments and compile to the concrete CNF semantics without out-of-range variables. Unit constraints, finite implications, and pairwise exactly-one groups have exact satisfaction equivalences; local programs have exact recursive clause counts, satisfiability reflection, and a proved well-scoped formula. All 75 declarations and nine reviewed theorem types have empty axiom closure.                                                                                                                                                                                            | This is a local clause compiler only. It does not enumerate the verifier tableau, encode initialization, transition windows, certificate length, or acceptance as a complete formula; prove an all-input emitter size/runtime polynomial; define a reduction to CNFSAT; or establish CNFSAT NP-completeness, CNFSAT in P, or $P = NP$ . |

| Milestone                                              | Status                        | Exact scope                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Boundary                                                                                                                                                                                                                                                                                                                                                          |
|--------------------------------------------------------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Finite whole-tableau Cook-Levin CNF syntax             | formalized-foundation-only    | A uniform bounded verifier-tableau problem is compiled answer-independently into one finite, well-scoped concrete CNF formula. The program emits one-hot symbol/head/state rows, first-match control updates, untouched-cell preservation, exact input-only or symbolic paired-certificate initialization, and a designated final accept constraint. Canonical encoding/decoding, exact local satisfaction reflection, and exact emitted clause counting are proved. All 81 declarations and ten reviewed theorem types have empty axiom closure. | This is finite formula syntax and local-clause reflection only. It does not yet prove that formula satisfiability is equivalent to an accepting raw verifier execution, prove external encoded-output-size or construction-runtime polynomials, define a concrete polynomial reduction to CNFSAT, or establish CNFSAT NP-completeness, CNFSAT in P, or $P = NP$ . |
| Typed direct-wire NAND semantics                       | formalized                    | Typed topological Boolean NAND programs and ordered multi-output direct-wire semantics.                                                                                                                                                                                                                                                                                                                                                                                                                                                           | This does not establish circuit minimization, SAT, or $P = NP$ .                                                                                                                                                                                                                                                                                                  |
| Finite enumeration, equivalence, and reference minimum | formalized                    | Exhaustive finite Boolean direct-wire search under the empty-profile reference model.                                                                                                                                                                                                                                                                                                                                                                                                                                                             | No polynomial-runtime result is obtained from this exhaustive reference search.                                                                                                                                                                                                                                                                                   |
| Concrete framed replacement and slack                  | formalized                    | The concrete serial framed context with support outputs and explicit bypass wires.                                                                                                                                                                                                                                                                                                                                                                                                                                                                | This is not an arbitrary-support replacement theorem for the report's global family.                                                                                                                                                                                                                                                                              |
| Locked-NAND local candidates and baseline accounting   | formalized                    | Typed local macro candidates, source-derived counts, and five finite local square baselines.                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Local minima are not a global BaselineDistinct or locked-NAND threshold theorem.                                                                                                                                                                                                                                                                                  |
| Conditional locked-NAND threshold boundary             | formalized-with-premises      | A proof-bearing six-premise candidate boundary for an arbitrary satisfiable proposition.                                                                                                                                                                                                                                                                                                                                                                                                                                                          | The premises are not instantiated by a uniform SAT-to-locked-NAND builder.                                                                                                                                                                                                                                                                                        |
| Fail-closed explicit-list residual routes              | formalized-explicit-list-only | Executable strict-gain search over a caller-supplied finite implementation list.                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Unresolved excludes no gain outside the supplied list and cannot imply ZeroSlack.                                                                                                                                                                                                                                                                                 |
| Global locked-NAND construction and threshold          | not-formalized                | A uniform answer-independent SAT instance builder, global baseline, trace equivalence, and threshold.                                                                                                                                                                                                                                                                                                                                                                                                                                             | The conditional boundary does not discharge this missing theorem.                                                                                                                                                                                                                                                                                                 |
| Global ZeroSlack, PCCMin, and polynomial runtime       | not-formalized                | Complete residual routing, global ZeroSlack contradiction, exact minimization, and polynomial bounds.                                                                                                                                                                                                                                                                                                                                                                                                                                             | Proof-bearing result structures and explicit-list failure do not supply global completeness.                                                                                                                                                                                                                                                                      |

| Milestone                                                   | Status         | Exact scope                                                                                                                           | Boundary                                                                                                                                                                                                                                                                                                                |
|-------------------------------------------------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Concrete standard<br>P-versus-NP target<br>and root theorem | not-formalized | Raw-machine-linked<br>complexity classes,<br>concrete SAT completeness<br>and a SAT decider, plus<br>the publication root<br>theorem. | The concrete target<br>definition remains inactive:<br>CNF-SAT now has a<br>raw-machine verifier and<br>NP-membership proof, but<br>a deterministic<br>polynomial-time CNF-SAT<br>decider, NP-completeness<br>transport, and<br>PNP.Main.p_eq_np<br>remain absent; the legacy<br>string-handle bridge is<br>ineligible. |

## 4 Compiled inventory summary

The inventory contains 6007 exported `PNP.*` kernel declarations from 56 modules. It excludes 1042 private compiler auxiliaries. Counts describe the compiled environment; they do not measure mathematical completeness.

| Module                                        | Declarations | Theorems |
|-----------------------------------------------|--------------|----------|
| PNP.Bridge                                    | 93           | 22       |
| PNP.Complexity                                | 93           | 20       |
| PNP.Concrete.BitString                        | 123          | 60       |
| PNP.Concrete.CNF                              | 247          | 60       |
| PNP.Concrete.CNFVerifier                      | 10           | 7        |
| PNP.Concrete.CNFWorkCorrectness               | 859          | 525      |
| PNP.Concrete.CNFWorkFrameCorrectness          | 16           | 4        |
| PNP.Concrete.CNFWorkInput                     | 29           | 14       |
| PNP.Concrete.CNFWorkMachine                   | 85           | 3        |
| PNP.Concrete.CNFWorkTransitions               | 64           | 63       |
| PNP.Concrete.CNFWorkUniversalCorrectness      | 294          | 145      |
| PNP.Concrete.Complexity                       | 312          | 92       |
| PNP.Concrete.CookLevinLayout                  | 204          | 71       |
| PNP.Concrete.CookLevinLocalCNF                | 172          | 53       |
| PNP.Concrete.CookLevinTableau                 | 71           | 23       |
| PNP.Concrete.CookLevinTableauCNF              | 179          | 60       |
| PNP.Concrete.CookLevinVerifierTableau         | 58           | 25       |
| PNP.Concrete.Machine                          | 284          | 67       |
| PNP.Concrete.PipelineCompiler                 | 38           | 27       |
| PNP.Concrete.PipelineInputFramer              | 106          | 18       |
| PNP.Concrete.PipelineMachineSimulation        | 175          | 79       |
| PNP.Concrete.PipelineOutputHandoff            | 23           | 4        |
| PNP.Concrete.PipelinePairedCompiler           | 29           | 25       |
| PNP.Concrete.PipelineRefinement               | 68           | 24       |
| PNP.Concrete.PipelineSequentialCompiler       | 36           | 28       |
| PNP.Concrete.PipelineSequentialStateNamespace | 30           | 21       |
| PNP.Concrete.PipelineStageBridges             | 77           | 59       |
| PNP.Concrete.PipelineStateNamespace           | 77           | 34       |
| PNP.Concrete.PipelineTapeGeometry             | 20           | 12       |
| PNP.Concrete.PipelineTerminalBridge           | 73           | 62       |
| PNP.Concrete.TapeBlankEquivalence             | 25           | 17       |
| PNP.Concrete.TapeHandoff                      | 18           | 11       |
| PNP.Concrete.Target                           | 2            | 1        |
| PNP.Concrete.TerminalOutputPacker             | 111          | 36       |
| PNP.Concrete.WorkInput                        | 23           | 14       |
| PNP.Concrete.WorkMachine                      | 308          | 108      |
| PNP.DirectWireBaseline                        | 48           | 25       |
| PNP.LockedNAND                                | 11           | 4        |
| PNP.LockedNANDBaseline                        | 57           | 22       |
| PNP.LockedNANDDirect                          | 84           | 57       |

| Module                          | Declarations | Theorems |
|---------------------------------|--------------|----------|
| PNP.LockedNANDLocalBaseline     | 30           | 22       |
| PNP.LockedNANDMacros            | 288          | 98       |
| PNP.LockedNANDPrefix            | 84           | 40       |
| PNP.LockedNANDThresholdBoundary | 60           | 35       |
| PNP.Main                        | 36           | 12       |
| PNP.NANDComposition             | 77           | 38       |
| PNP.NANDEnumerator              | 120          | 46       |
| PNP.NANDMinimum                 | 56           | 23       |
| PNP.NANDSemantics               | 198          | 80       |
| PNP.NANDSlack                   | 15           | 12       |
| PNP.NANDTruthTable              | 72           | 28       |
| PNP.PCCMin                      | 44           | 8        |
| PNP.ResidualBand                | 9            | 2        |
| PNP.ResidualRoutes              | 141          | 46       |
| PNP.SAT                         | 4            | 1        |
| PNP.ZeroSlack                   | 141          | 21       |

## 5 Remaining formal blockers

Seven formal blockers remain active:

1. `Formal.ConcreteSAT`
2. `Formal.LockedNANDThreshold`
3. `Formal.ResidualBandMinimizer`
4. `Formal.ZeroSlack`
5. `Formal.PolynomialRuntimeAndCertificateBounds`
6. `Formal.RootTheoremAndAxiomAudit`

The conditional locked-NAND threshold module assumes typed baseline and full candidates, baseline conditions, preservation of existing outputs, an unsatisfiable final-zero law, and satisfiable final-output laws. The explicit residual scanner searches only a caller-supplied finite list. Neither result constructs the missing global reduction or proves global ZeroSlack, polynomial PCCMin, SAT in P, or P equals NP.

## 6 Historical report provenance

Earlier 56-page report revisions stated a direct checker-mediated P-versus-NP claim. Those bytes remain historical audit material at their pinned legacy coordinates and through the explicit archive and supersession records. They are not imported into, quoted as authority by, or used to generate this report. File identity and replay of historical predicates do not establish the named mathematical propositions.

Source tag            `final-pnp-proof-report-hardened-7072f8d`  
Source commit        `7072f8d0bda6d44d240f9bb3fad624fd357e1278`  
Archive manifest      `archive/legacy-v0/ARCHIVE.json`

## 7 Verification

The current reconstruction can be checked with:

```
lake build PNP
node scripts/export-lean-theorem-inventory.mjs --check
node scripts/generate-formal-publication.mjs --check
node pcc-formal-reconstruction-status0.mjs --json
npm run pnp:verify -- --no-write
npm run report:check
```

The inventory coordinate is PNP-LEAN-THEOREM-INVENTORY-2026-07-14-37. Its exact byte digest is 7e6aac322da8d6bb8db813217daf6411fae937cd101f3b6e960cefeb56330590. The report is regenerated from that inventory and the reviewed publication map. The reviewed milestone source-closure SHA-256 is 1a4b7aee6800b4eee817d1e87ee14f9f49c8df7bb9055525d0dd821b211461da; the computed and pinned values match. Successful regeneration confirms consistency of these status surfaces; it does not turn an absent concrete theorem into a proof.